Assignment 1

CSA0903-Java Programming

Name: Sowmiya. D

Reg No.:192511063

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate Changed Requirement

The university introduces **two new vehicle categories** – e-bikes (charged a flat fee) and delivery vans (charged at double the car rate during 9 AM – 5 PM). Visitors may now buy a **one-day pass**. The facility is extended from 3 to **5 floors** with a different number of slots on each floor.

Re-evaluate your original design in response to these changed requirements. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks Discuss:

- What has changed from the original problem
- Which of your classes, methods and data structures are affected
- Why a different number of slots on each floor affects your array design
- Which classes, if any, would have to be modified rather than extended

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original design remains appropriate. Discuss:

- Whether new vehicle types can be added only by writing new subclasses
- Whether the pricing rules are spread across many classes or kept in one place
- Whether the existing tested classes can remain unchanged

Evidence Expected

A technically supported evaluation of the original design under the new requirements.

C. Adaptation or Redesign – 2 Marks

Decide whether to retain, modify or redesign the design. If you modify or redesign it:

- Describe the revised class structure (for example: a jagged array for floors, a Pass interface, a PricingRule abstraction)
- Explain what has changed
- Implement the new vehicle categories and the one-day pass

If you retain the original design, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response with clear object-oriented reasoning and working code.

D. Evidence-Based Defence – 2 Marks Defend your

final design decision using:

- The number of existing classes that had to change
- The ease of adding a further vehicle type in future
- Test results for the new categories
- Relevant trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating object-oriented principles, test evidence and the changed requirements.

Solution:

TASK 12 – Re-evaluate and Adapt the Design

Introduction

The university has introduced new requirements for the Campus Smart-Parking System. Two new vehicle categories, e-bikes and delivery vans, must be supported. Visitors can purchase a one-day parking pass. The parking facility has also expanded from three floors to five floors, with different slot capacities on each floor.

The original design must therefore be modified to support different pricing methods, visitor passes and unequal floor capacities. I recommend modifying the existing design instead of completely redesigning it because inheritance and polymorphism can still be reused.

A. Impact Analysis

1. Changes in requirements

Original system	**New requirement**
Existing vehicle categories	Add e-bikes and delivery vans
Hourly parking charges	Flat e-bike fee and time-based van fee
No visitor pass	One-day visitor pass
Three floors	Five floors
Similar slot arrangement	Different slot capacity on each floor

2. Affected classes and methods

- **Vehicle:** Acts as the common superclass for the new vehicle categories.
- **EBike:** Calculates a fixed parking fee.
- **DeliveryVan:** Charges double the normal car rate during 9 AM–5 PM.
- **VisitorPass:** Stores visitor registration and one-day pass fee.
- **ParkingTicket:** Supports both normal parking tickets and visitor passes.
- **ParkingLot:** Handles five floors, slot allocation, vehicle entry and exit.

3. Why the array design must change

The original two-dimensional array assumes the same number of slots on every floor. This wastes space when floors have different capacities.

A **jagged array** is more suitable because each floor can have a different number of slots.

Example:

```
ParkingTicket[][] floors = new ParkingTicket[5][];


floors[0] = new ParkingTicket[2];

floors[1] = new ParkingTicket[3];

floors[2] = new ParkingTicket[1];

floors[3] = new ParkingTicket[2];
```

floors[4] = new ParkingTicket[4];

Each floor stores only the number of slots it actually needs.

B. Evaluation of the Original Design

The original design uses inheritance and polymorphism, so new vehicle types can be added as subclasses. However, the original hourly fee calculation is not sufficient for flat fees and time-dependent charges.

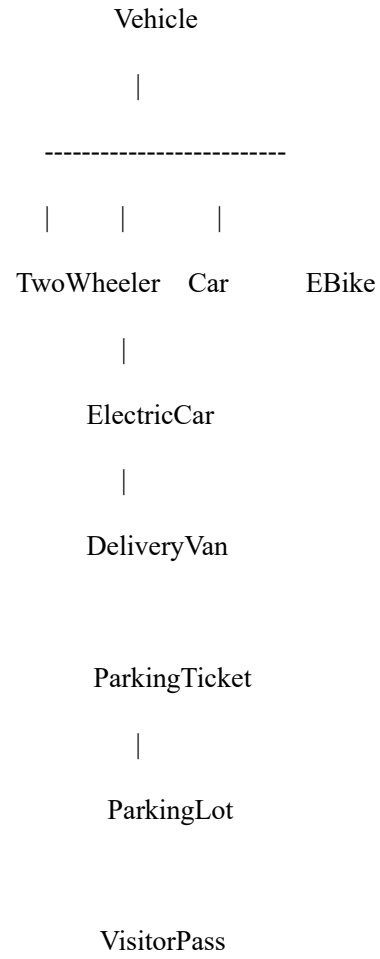
The Vehicle hierarchy can be extended with EBike and DeliveryVan. The existing TwoWheeler and Car pricing behaviour can remain unchanged. ParkingTicket must be modified to support visitor passes, and ParkingLot must be modified to support five floors.

Keeping pricing behaviour inside the respective vehicle classes avoids spreading vehicle-specific pricing conditions throughout the parking system.

C. Adaptation and Redesigned Class Structure

Decision: Modify the existing design

The revised class structure is:



The main changes are:

1. Add EBike with a flat fee of ₹20 per visit.

2. Add DeliveryVan with a rate of ₹60 per hour during 9 AM–5 PM and ₹30 per hour outside that period.
3. Add VisitorPass with a one-day fee of ₹100.
4. Replace the equal-sized floor array with a jagged array.
5. Modify parking entry and exit methods to handle different floor capacities and ticket types.

Pricing assumptions used in the implementation: The delivery-van rate is selected using the entry time. The e-bike fee is ₹20 per visit. The visitor pass costs ₹100 for one parking visit during the pass day.

Complete Java implementation

Save as Main.java in your Task12 folder. This is a single-file implementation.

```
import java.time.LocalTime;


// Abstract superclass
abstract class Vehicle {
   private final String regNo;


   public Vehicle(String regNo) {
      if (regNo == null || regNo.trim().isEmpty()) {
         throw new IllegalArgumentException("Invalid registration");
      }
      this.regNo = regNo;
   }
   public String getRegNo() {
      return regNo;
   }
   public abstract double calculateFee(int minutes, LocalTime entryTime);
}


// Existing vehicle categories
```

```
class TwoWheeler extends Vehicle {
    public TwoWheeler(String regNo) {
        super(regNo);
    }

    @Override
    public double calculateFee(int minutes, LocalTime time) {
        return minutes == 0 ? 0 : ((minutes + 59) / 60) * 10;
    }
}
class Car extends Vehicle {
    public Car(String regNo) {
        super(regNo);
    }
    @Override
    public double calculateFee(int minutes, LocalTime time) {
        return minutes == 0 ? 0 : ((minutes + 59) / 60) * 30;
    }
}
class ElectricCar extends Car {
    public ElectricCar(String regNo) {
        super(regNo);
    }
}
// New e-bike category
class EBike extends Vehicle {
    public EBike(String regNo) {
        super(regNo);
```

```
    }
    @Override
    public double calculateFee(int minutes, LocalTime time) {
        return 20; // Flat fee per visit
    }
}
// New delivery van category
class DeliveryVan extends Vehicle {
    public DeliveryVan(String regNo) {
        super(regNo);
    }
    @Override
    public double calculateFee(int minutes, LocalTime entryTime) {
        if (minutes == 0) return 0;
        int hours = (minutes + 59) / 60;
        boolean peakTime =
            !entryTime.isBefore(LocalTime.of(9, 0)) &&
            entryTime.isBefore(LocalTime.of(17, 0));

        double rate = peakTime ? 60 : 30;

        return hours * rate;
    }
}

// One-day visitor pass
class VisitorPass {
    private final String registration;
```

```
    private final double fee = 100;


    public VisitorPass(String registration) {
        this.registration = registration;
    }
    public String getRegistration() {
        return registration;
    }
    public double getFee() {
        return fee;
    }
}
// Parking ticket
class ParkingTicket {
    private final Vehicle vehicle;
    private final VisitorPass pass;
    private final int floor;
    private final int slot;
    public ParkingTicket(Vehicle vehicle, int floor, int slot) {
        this(vehicle, null, floor, slot);
    }
    public ParkingTicket(Vehicle vehicle, VisitorPass pass,
                         int floor, int slot) {
        this.vehicle = vehicle;
        this.pass = pass;
        this.floor = floor;
        this.slot = slot;
    }
```

```
    public String getRegistration() {
        return vehicle.getRegNo();
    }
    public int getFloor() {
        return floor;
    }
    public int getSlot() {
        return slot;
    }

    public double calculateFee(int minutes, LocalTime time) {
        if (minutes < 0) {
            throw new IllegalArgumentException("Invalid duration");
        }

        if (pass != null) {
            return pass.getFee();
        }

        return vehicle.calculateFee(minutes, time);
    }
}

// Parking lot with different capacities on each floor
class ParkingLot {
    private final ParkingTicket[][] floors;

    public ParkingLot(int[] capacities) {
```

```
    if (capacities == null || capacities.length != 5) {
        throw new IllegalArgumentException(
            "Parking lot must have five floors");
    }

    floors = new ParkingTicket[5][];

    for (int i = 0; i < 5; i++) {
        if (capacities[i] <= 0) {
            throw new IllegalArgumentException(
                "Invalid floor capacity");
        }

        floors[i] = new ParkingTicket[capacities[i]];
    }
}

private ParkingTicket find(String regNo) {
    for (ParkingTicket[] floor : floors) {
        for (ParkingTicket ticket : floor) {
            if (ticket != null &&
                ticket.getRegistration().equalsIgnoreCase(regNo)) {
                return ticket;
            }
        }
    }
    return null;
}
```

```
public boolean enter(Vehicle vehicle, int floor, int slot) {
    if (vehicle == null ||
        floor < 1 || floor > floors.length ||
        slot < 1 || slot > floors[floor - 1].length) {
        return false;
    }
    if (find(vehicle.getRegNo()) != null ||
        floors[floor - 1][slot - 1] != null) {
        return false;
    }
    floors[floor - 1][slot - 1] =
        new ParkingTicket(vehicle, floor, slot);

    return true;
}

public boolean enterVisitor(String regNo, int floor, int slot) {
    if (regNo == null || regNo.trim().isEmpty() ||
        floor < 1 || floor > floors.length ||
        slot < 1 || slot > floors[floor - 1].length ||
        find(regNo) != null ||
        floors[floor - 1][slot - 1] != null) {
        return false;
    }
    Vehicle visitorVehicle = new Car(regNo);
    VisitorPass pass = new VisitorPass(regNo);
```

```
        floors[floor - 1][slot - 1] =
            new ParkingTicket(visitorVehicle, pass, floor, slot);

        return true;
    }

    public double exit(String regNo, int minutes, LocalTime time) {
        if (minutes < 0 || time == null) {
            return -1;
        }

        ParkingTicket ticket = find(regNo);

        if (ticket == null) {
            return -1;
        }

        double fee = ticket.calculateFee(minutes, time);

        floors[ticket.getFloor() - 1]
              [ticket.getSlot() - 1] = null;

        return fee;
    }
}

// Main class
public class Main {
```

```
public static void main(String[] args) {

    // Five floors with different slot capacities
    ParkingLot lot = new ParkingLot(
        new int[]{2, 3, 1, 2, 4}
    );
    System.out.println("CAMPUS SMART PARKING SYSTEM");
    System.out.println();
    // E-bike test
    System.out.println("E-Bike Entry: " +
        lot.enter(new EBike("EB101"), 1, 1));
    System.out.println("E-Bike Fee: Rs. " +
        lot.exit("EB101", 90, LocalTime.of(10, 0)));
    // Delivery van test during peak time
    System.out.println("Delivery Van Entry: " +
        lot.enter(new DeliveryVan("DV201"), 2, 1));
    System.out.println("Delivery Van Fee: Rs. " +
        lot.exit("DV201", 60, LocalTime.of(10, 0)));
    // Delivery van test outside peak time
    lot.enter(new DeliveryVan("DV202"), 2, 2);
    System.out.println("Van Outside Peak Fee: Rs. " +
        lot.exit("DV202", 60, LocalTime.of(18, 0)));
    // Visitor pass test
    System.out.println("Visitor Entry: " +
        lot.enterVisitor("VIS301", 5, 1));
    System.out.println("Visitor Pass Fee: Rs. " +
        lot.exit("VIS301", 120, LocalTime.of(11, 0)));
    // Invalid floor test
```

```
        System.out.println("Invalid Floor Entry: " +
            lot.enter(new Car("CAR401"), 6, 1));
        // Duplicate registration test
        lot.enter(new Car("CAR501"), 3, 1);
        System.out.println("Duplicate Registration: " +
            lot.enter(new Car("CAR501"), 4, 1));
        System.out.println();
        System.out.println("All tests completed.");
    }
}
```

Run the program

Open the Task12 folder in VS Code terminal and execute:

javac Main.java

java Main

D. Evidence-Based Defence

1. Expected program output

CAMPUS SMART PARKING SYSTEM

E-Bike Entry: true

E-Bike Fee: Rs. 20.0

Delivery Van Entry: true

Delivery Van Fee: Rs. 60.0

Van Outside Peak Fee: Rs. 30.0

Visitor Entry: true

Visitor Pass Fee: Rs. 100.0

Invalid Floor Entry: false

Duplicate Registration: false

All tests completed.

2. Test results

Test case	Expected result
E-bike, 90 minutes	₹20
Delivery van at 10 AM	₹60
Delivery van at 6 PM	₹30
Visitor day pass	₹100
Invalid floor	Rejected
Duplicate registration	Rejected

3. Design evaluation

- Five existing classes are adapted: Vehicle, TwoWheeler, Car, ParkingTicket and ParkingLot.
- Two new vehicle subclasses are introduced: EBike and DeliveryVan.
- VisitorPass is introduced to manage visitor parking charges.
- Future vehicle categories can be added by extending Vehicle and implementing calculateFee().
- The jagged array supports different capacities on all five floors.

4. Advantages and limitations

Advantages:

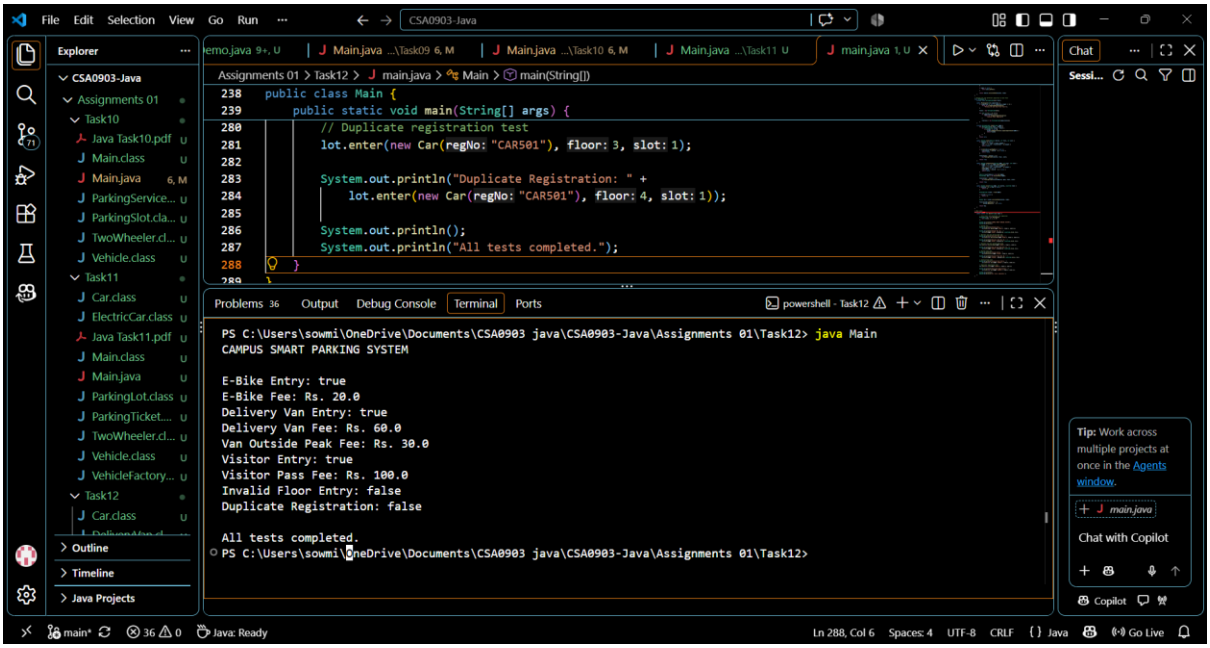
- Reuses the original inheritance structure.
- Supports different vehicle pricing methods.
- Reduces wasted array space.
- Supports visitor passes and five floors.

Limitations:

- Delivery-van pricing uses entry time rather than calculating the exact time spent during peak hours.
- The current implementation does not store the actual pass date or enforce expiry at midnight.

Conclusion

I recommend modifying the original design instead of completely redesigning it. The revised implementation supports new vehicle categories, visitor passes and different floor capacities while retaining inheritance, encapsulation, abstraction and polymorphism. The test cases demonstrate the expected pricing and validation behaviour.


File Edit Selection View Go Run
CSA0903-Java
Explorer
CSA0903-Java
Assignments 01
Task10
Java Task10.pdf
Main.class
Main.java
ParkingService...
ParkingSlot.cla...
TwoWheeler.cl...
Vehicle.class
Task11
Car.class
ElectricCar.class
Java Task11.pdf
Main.class
Main.java
ParkingLot.class
ParkingTicket....
TwoWheeler.cl...
Vehicle.class
VehicleFactory...
Task12
Car.class
Outline
Timeline
Java Projects
Main.java ...\Task09 6, M
Main.java ...\Task10 6, M
Main.java ...\Task11 U
main.java 1, U
Assignments 01 > Task12 > main.java > Main > main(String[])
238 public class Main {
239 public static void main(String[] args) {
280 // Duplicate registration test
281 lot.enter(new Car(regNo: "CAR501"), floor: 3, slot: 1);
282
283 System.out.println("Duplicate Registration: " +
284 lot.enter(new Car(regNo: "CAR501"), floor: 4, slot: 1));
285
286 System.out.println();
287 System.out.println("All tests completed.");
288 }
Problems 36
Output
Debug Console
Terminal
Ports
powershell - Task12
PS C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task12> java Main
CAMPUS SMART PARKING SYSTEM
E-Bike Entry: true
E-Bike Fee: Rs. 20.0
Delivery Van Entry: true
Delivery Van Fee: Rs. 60.0
Van Outside Peak Fee: Rs. 30.0
Visitor Entry: true
Visitor Pass Fee: Rs. 100.0
Invalid Floor Entry: false
Duplicate Registration: false
All tests completed.
PS C:\Users\sowmi\OneDrive\Documents\CSA0903 java\CSA0903-Java\Assignments 01\Task12>
Chat
Tip: Work across multiple projects at once in the Agents window.
main.java
Chat with Copilot
Copilot
main*
36 0
Java: Ready
Ln 288, Col 6
Spaces: 4
UTF-8
CRLF
Java
Go Live